Un guide gastronomique

Compétences développées dans le module

- Ecrire des requêtes d'interrogation à une table
  - Projections simples
  - Restrictions
- Ecrire des requêtes en affichant des résultats parlants pour l'utilisateur : renommage des colonnes de la table de résultat, tri des résultats.
- Ecrire des requêtes d'interrogation complexes à plusieurs tables
  - Jointures
- Faire des calculs sur une base de données
  - Avec des fonctions d'agrégation sans partition
  - Avec partition
- Ecrire des requêtes reposant sur des sous-requêtes
  - IN et NOT IN

Le syndicat d'initiative d'une grande ville a décidé de constituer un guide gastronomique des restaurants de la cité.
Ce guide doit renseigner les résidents, habitants et touristes sur les différents plats, et tout particulièrement les spécialités, que l'on peut déguster en ville et sur les vins mis en vente dans les restaurants.

La base de données permet d'avoir :

- Les **restaurants** de la ville, enregistrés avec un code d'identification, un nom, une adresse, les jours d'ouverture, leurs horaires d'ouverture à midi et le soir, la liste des plats proposés à la carte et la liste des vins proposés.
- Les **plats** que l'on peut déguster en ville. Pour chaque plat, on doit connaître son nom, sa place dans le repas (entrée, dessert, ...), la liste des prix dans les restaurants qui le proposent à leur carte, le nom éventuel du vin idéal (d'après un grand gastronome) et une couleur de vin recommandée permettant de choisir un vin adapté dans le cas où l'on ne peut prendre le vin idéal. Cette couleur peut être différente de la couleur du vin idéal.
- Les **vins** que l'on peut trouver en ville avec, pour chacun d'eux, des informations sur leur nom, la date de récolte du raisin, la région d'origine, la couleur, et la liste des prix dans les restaurants qui le proposent.

Voici le schéma relationnel qui a été proposé pour cette base :

Restaurants (<u>code</u>, nom, ville)
Vins (<u>code</u>, cru, millesime, region, couleur)
Plats (<u>code</u>, nom, place, couleurVinRdee, #codeVinIdeal)
ServirVin (<u>#codeRestau, #codeVin</u>, prixVin)
ServirPlat (<u>#codeRestau, #codePlat</u>, prixPlat, specialite)
Ouverture (<u>#codeRestau</u>, <u>libJour</u>, <u>horaireOuverture</u>, horaireFermeture)

Ecrire les requêtes SQL qui permettent de répondre aux questions ci-dessous.

TD4 et TP4 : Jointures, fonctions d'agrégation et partitions

La jointure permet d'associer les lignes de tables sur le critère d'égalité entre un attribut de chaque table, ces attributs devant être de même type et domaine.

Nous ne parlerons ici que des jointures intérieures (JOIN ou INNER JOIN), c'est-à-dire des jointures pour lesquelles seules les lignes dont les attributs correspondent seront renvoyées. JOIN est par défaut un INNER JOIN.

- Il existe deux façons de faire des jointures.

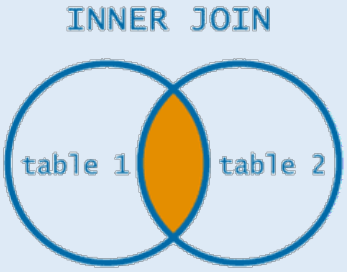
En utilisant la Clause JOIN dans le FROM

```
SELECT nomColonne1, ..., nomColonneN
FROM table1 JOIN table2 ON
table1.colonneX=table2.colonneY;
```

En utilisant la Clause INNER JOIN dans le FROM

```
SELECT nomColonne1, ..., nomColonneN
FROM table1 INNER JOIN table2 ON
table1.colonneX=table2.colonneY;
```

Ces requêtes affichent les attributs (nomColonne1, ..., nomColonneN) pour les lignes **qui ont une correspondance** entre table1 et table2 sur le **critère d'égalité** entre colonneX de table 1 et colonneY de table 2.



Remarque : Si des colonnes portent exactement le même nom dans les deux tables, il est obligatoire de préciser à quelle table appartient chaque colonne lors de leur utilisation (dans les Clauses SELECT, WHERE, ORDER BY…).
Cela peut se faire soit à l'aide du **nom complet** de la table, soit en utilisant un **alias**. Cette précision permet d'éviter les erreurs d'ambiguïté lors de l'exécution de la requête. **Par exemple,** si table1 et table2 avaient toutes les deux une colonne nommée colonneX :

Exemple : En utilisant le nom de la table

```
SELECT table1.nom
FROM table1 JOIN table2 ON
table1.colonneX=table2.colonneX;
```

Exemple : En utilisant l'alias de la table

```
SELECT t1.nom
FROM table1 t1 JOIN table2 ON
t1.colonneX =t2.colonneX;
```

3) Jointures

Q12. Noms des plats qu'il est conseillé de consommer avec un "Gewurztraminer" ?

```
SELECT P.nom as nomPlat
FROM Plats P
INNER JOIN Vins V ON P.codeWinIdeal = V.code
WHERE V.cru = 'Gewurztraminer';
```

Q13. Cru, couleur et millésime du vin recommandé avec un "Plateau de fruits de mer" ?

```
SELECT V.cru, V.couleur, V.millesime
FROM Plats P
INNER JOIN Vins V ON P.codeWinIdeal = V.code
WHERE P.nom = 'Plateau de fruits de mer';
```

Q14. Noms et ville des restaurants qui proposent du "Cassoulet" à moins de 80 euros ?

```
SELECT R.nom AS nomRestau, R.ville
FROM Restaurants R
INNER JOIN ServirPlat S ON S.codeRestau = R.code
INNER JOIN Plats P ON S.codePlat = P.code
WHERE P.nom = 'Cassoulet'
AND S.prixPlat < 80;
```

Q15. Noms des restaurants qui servent de l'Irouléguy à moins de 50 euros ?

```
SELECT DISTINCT R.nom AS nomRestau, R.ville
FROM Restaurants R
INNER JOIN ServirVin S ON S.codeRestau = R.code
INNER JOIN Vins V ON S.codeVin = V.code
WHERE V.cru = 'Irouléguy'
AND S.prixVin < 50;
```

4) Agrégation

Les fonctions d'agrégation en SQL permettent de faire des calculs statistiques sur les enregistrements d'une colonne.

- AVG() pour calculer la moyenne sur un ensemble d'enregistrements
- COUNT() pour compter le nombre d'enregistrement sur une table ou une colonne distincte
- MAX() pour récupérer la valeur maximale d'une colonne sur un ensemble de lignes. Cela s'applique à la fois pour des données numériques qu'aux données alphanumériques
- MIN() pour récupérer la valeur minimum de la même manière que MAX()
- SUM() pour calculer la somme sur un ensemble d'enregistrements

Syntaxe : `SELECT fonction(colonne) FROM table`

Q16. Prix minimum d'un vin ?

```
SELECT MIN(prixVin) AS prixMin
FROM ServirVin;
```

Q17. Nombre de vins rouges ?

```
SELECT COUNT(*) AS nbRouge
FROM Vins
WHERE couleur = 'Rouge';
```

Q18. Prix moyen d'un vin blanc ?

```
SELECT AVG(S.prixVin) AS prixMoyen
FROM Vins V
INNER JOIN ServirVin S ON S.codeVin = V.code
AND V.couleur = 'Blanc';
```

Q19. Prix moyen, minimale et maximal d'un vin rouge ?

```
SELECT AVG(S.prixVin) AS prixMoyen, MIN(S.prixVin) AS prixMin, MAX(S.prixVin) AS prixMax
FROM Vins V
INNER JOIN ServirVin S ON S.codeVin = V.code
WHERE V.couleur = 'Rouge';
```